



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO
AO ENSINO MÉDIO

RAFAEL GOMES EMILIANO

KAIQUE AVANTE

PYETRO NUNES

SAVIO PEREIRA

TURMA 213

Plano de Segurança da Informação Sabor e Clic

SÃO PAULO 2026

RAFAEL GOMES EMILIANO

KAIQUE AVANTE

PYETRO NUNES

SAVIO PEREIRA

TURMA 213

Plano de Segurança da Informação Sabor e Clic

Trabalho apresentado no
curso Técnico em
Desenvolvimento de Sistemas
integrado ao Ensino Médio do
Instituto Federal de São Paulo.
Professor(a)(s): Claudete Alves,
Greice Antolini e Matheus Pereira

SÃO PAULO 2026

SUMÁRIO

1 OBJETIVO E DELIMITAÇÃO	4
1.1 Escopo funcional e limites de confiança	4
1.2 Princípios adotados	5
2 MATRIZ DE CONTROLE DE ACESSO (RBAC)	5
2.1 Operações CRUD previstas	6
2.2 Aplicação lógica da autorização	6
3 REGRAS DE VALIDAÇÃO E FEEDBACK.....	7
3.1 Padrão de respostas	7
4 PROTEÇÕES POR CATEGORIA DA OWASP TOP 10:2021.....	8
5 PROTEÇÕES COMPLEMENTARES E ORDEM DE IMPLEMENTAÇÃO .	13
5.1 CSRF e XSS.....	13
5.2 Etapas e resultados esperados	13
5.3 Casos mínimos de teste para apresentar	13
6 LIMITAÇÕES CONHECIDAS DA VERSÃO ATUAL.....	14
7 NOTA DE VERSÃO DA OWASP	14
8 REFERÊNCIAS	15

1 OBJETIVO E DELIMITAÇÃO

Este documento define como o Sabor & Clic pretende proteger usuários, reservas, sessões de atendimento, pedidos, cardápio e estoque. Apresenta políticas de autorização, validações de entrada, feedback previsto, controles por categoria da OWASP Top 10 e formas de comprovar sua eficácia na próxima etapa de desenvolvimento.

No 3º bimestre, a entrega de Segurança contempla a matriz de permissões (RBAC), as regras de validação no JavaScript e a documentação das políticas e mensagens. A implementação completa da autenticação Flask, autorização obrigatória no servidor, persistência no MySQL e proteções de produção pertence ao 4º bimestre. Nenhuma medida descrita como “prevista” deve ser interpretada como já implementada.

Referência adotada. O enunciado da disciplina cita A03 — Injection e A07 — Identification and Authentication Failures, nomenclaturas da OWASP Top 10:2021. Por isso, a classificação principal deste plano segue a edição de 2021. A OWASP publicou uma edição 2025 com categorias e numeração alteradas; a diferença é indicada na seção 7.

1.1 Escopo funcional e limites de confiança

O fluxo de negócio é Reserva → Sessão → Pedido → KDS. O Cliente consulta bancadas, registra reservas e envia pedidos da própria sessão; o Cozinheiro visualiza a fila de preparo e atualiza seu andamento; o Administrador gerencia cadastros, recursos, usuários e relatórios.

A “sessão” do atendimento é a entidade do negócio, vinculada à reserva. Ela não é a sessão de autenticação HTTP do Flask. No sistema definitivo, a identidade do usuário será obtida exclusivamente da autenticação validada pelo backend; identificadores recebidos no JSON não serão prova de propriedade de uma reserva ou pedido.

O protótipo atual utiliza páginas HTML, JavaScript e localStorage, além de uma API Flask com dados em memória. A API demonstrativa ainda não autentica nem autoriza solicitações. A futura integração introduzirá a camada de persistência SQL escrita pela equipe, sem geração automática do esquema por ORM.

1.2 Princípios adotados

- Negação por padrão: toda operação não declarada na matriz de permissões será negada.
- Menor privilégio: cada perfil acessará apenas os dados e operações necessários para sua função.
- Defesa em profundidade: HTML/JavaScript orientam o preenchimento, mas o Flask repetirá as verificações antes de persistir qualquer dado.
- Identidade e propriedade: autorizar a rota não dispensa verificar se o recurso pertence ao solicitante.
- Erros seguros: respostas previsíveis para o usuário; detalhes técnicos ficarão restritos ao log do servidor.
- Integridade e rastreabilidade: alterações críticas serão validadas, registradas e, quando necessário, executadas em transações.

2 MATRIZ DE CONTROLE DE ACESSO (RBAC)

Os papéis persistidos no projeto são cliente, cozinha e admin. O cadastro público atribuirá exclusivamente o perfil cliente; somente um administrador autenticado poderá criar ou modificar perfis privilegiados. Conteúdos públicos não exigirão login, enquanto dados pessoais e operações que alteram estado exigirão autenticação e autorização no servidor.

Legenda: P = público; Próprio = recurso vinculado à identidade do solicitante; Todos = acesso operacional necessário; ADM = somente administrador; — = operação não permitida. Esta matriz descreve a política alvo, não a proteção atual da API.

Rota / operação	Cliente	Cozinha	Admin
POST /api/cadastro — criar conta	P (perfil cliente)	P (perfil cliente)	P (perfil cliente)
POST /api/login — autenticar	P	P	P
GET /api/cadastro/<id> — consultar cadastro	Próprio	Próprio	Todos
GET /api/bancadas; GET /api/cardapio	P	P	P
POST /api/reservas — criar	Próprio	—	Todos
GET /api/reservas/<id> — consultar	Próprio	—	Todos
POST /api/sesoes — iniciar atendimento	Próprio*	—	Todos
GET /api/sesoes/<id> — consultar	Próprio	—	Todos
POST /api/sesoes/<id>/encerrar	Próprio*	—	Todos
POST /api/pedido/cadastro — criar pedido	Próprio	—	Todos

Rota / operação	Cliente	Cozinha	Admin
GET /api/pedido/<id> — consultar	Próprio	Todos necessários	Todos
GET /api/kds; GET /api/kds/<id>	—	Todos necessários	Todos

* O início e o encerramento pelo cliente somente serão autorizados para sua própria reserva e se as condições do atendimento forem satisfeitas (horário válido, sessão não encerrada, ausência de pendências incompatíveis). O grupo poderá restringir essas operações ao administrador caso a regra de negócio seja alterada; a documentação e a API devem permanecer consistentes.

2.1 Operações CRUD previstas

Recurso	Cliente	Cozinha	Administrador
Perfil e usuários	Ler/editar próprios dados	Ler/editar próprios dados	CRUD de usuários e papéis*
Reservas	Criar/ler/alterar/cancelar próprias**	—	CRUD operacional
Sessões	Consultar/iniciar/encerrar própria**	Consultar contexto mínimo do pedido	Gerenciar
Pedidos	Criar e ler próprios; cancelar se regra permitir	Ler e atualizar preparo	Consultar/gerenciar
Produtos, bancadas e ingredientes	Ler dados públicos	Ler disponibilidade e estoque	CRUD
Substituições, relatórios e permissões	—	Consultar o estritamente necessário	CRUD / relatórios

* A atribuição de administrador deve ser controlada adicionalmente; não será permitido que um usuário eleve o próprio papel. ** Alterações de reserva e cancelamentos dependem de prazo, estado e regras de negócio. As rotas CRUD adicionais são planejadas; não são tratadas como existentes no protótipo.

2.2 Aplicação lógica da autorização

A checagem ocorrerá antes da operação do controller. Primeiro, identificar o usuário autenticado; segundo, verificar seu papel; terceiro, carregar o recurso; quarto, verificar vínculo/propriedade e estado. Caso qualquer verificação falhe, a execução será interrompida. O ID enviado pelo cliente será usado para localizar um objeto, nunca para definir a identidade de quem o solicita.

```
# Esquema conceitual, não é código integrado ao projeto:
usuario = obter_usuario_da_autenticacao()
if usuario is None: return erro(401)
reserva = buscar_reserva(id_reserva)
if reserva is None: return erro(404)
if usuario.perfil != "admin" and reserva.id_cliente != usuario.id:
    return erro(403)
```

Somente agora executar a ação permitida.

3 REGRAS DE VALIDAÇÃO E FEEDBACK

A validação de interface deve reduzir erros de preenchimento e apresentar mensagens claras. No backend, campos serão interpretados com tipos explícitos, limites de tamanho, regras de negócio e lista de valores admitidos. Dados inesperados não serão convertidos silenciosamente em permissões ou comandos; totais e preços serão calculados ou confirmados pelo servidor.

Dado	Regra no front-end	Regra definitiva no backend
E-mail	Campo obrigatório; formato de e-mail; retirar espaços externos.	Normalizar; validar tamanho/formato; UNIQUE no banco; não revelar cadastro no login.
Senha	Campo obrigatório; requisitos de criação e confirmação.	Verificar hash seguro; nunca armazenar em texto puro; limitar tentativas.
Data/horário	Data válida e futura; início anterior ao fim.	Converter para datetime, conferir período e conflito entre reservas.
Bancada/pessoas	Selecionar bancada; pessoas inteiras e positivas.	Confirmar existência, capacidade, disponibilidade e intervalo permitido.
Pedido/itens	Carrinho não vazio; quantidade inteira positiva.	Verificar sessão ativa e própria, disponibilidade e preços oficiais.
Observação, nome, telefone	Comprimentos máximos, campos obrigatórios e tipo apropriado.	Validar comprimento/estrutura; escapar saída; não interpolar em SQL.
Perfil, preço e total	Não expor edição arbitrária no formulário público.	Ignorar perfil/total do cliente; atribuir perfil e calcular valores no servidor.

3.1 Padrão de respostas

HTTP	Significado	Mensagem de interface
400	Campos ou formato inválidos	Verifique os dados informados.
401	Não autenticado / credenciais inválidas	Não foi possível autenticar sua solicitação.
403	Autenticado sem permissão	Você não tem permissão para realizar esta operação.
404	Recurso ausente ou deliberadamente ocultado	Recurso não encontrado.
409	Conflito de horário, estado ou duplicidade	A operação conflita com os dados atuais.
429	Tentativas excessivas	Aguarde alguns instantes antes de tentar novamente.
500	Erro inesperado	Ocorreu um erro. Tente novamente mais tarde.

Respostas JSON usarão um formato consistente, por exemplo {"erro": "...", "codigo": "RESERVA_CONFLITO"}. O frontend exibirá apenas a mensagem apropriada. Detalhes como SQL, stack traces, hash de senha e segredos nunca serão devolvidos ao navegador.

4 PROTEÇÕES POR CATEGORIA DA OWASP TOP 10:2021

Cada categoria apresenta o risco, sua aplicação específica ao Sabor & Clic, o controle de proteção previsto e uma evidência verificável. Os exemplos são orientações de implementação; este relatório não comprova a execução das medidas finais.

A01 — Broken Access Control (controle de acesso quebrado)

Risco no contexto do projeto. Um usuário pode realizar operações além das permissões previstas ou acessar dados de outra pessoa pela troca de um identificador.

Aplicação ao Sabor & Clic. O localStorage atual permite simular perfis; o cadastro demonstrativo aceita um campo de perfil e a API não valida a identidade real em suas rotas. Os recursos sensíveis são /api/reservas/<id>, /api/sessoes/<id>, /api/pedido/<id> e /api/kds.

Medidas previstas. Aplicar RBAC no Flask com negação por padrão; obter identidade do login validado; validar a propriedade de reservas, sessões e pedidos; impedir autoconcessão de papéis; limitar os dados de resposta por perfil; não confiar em botões ocultos nem em IDs fornecidos pelo navegador.

Verificação e evidência. Testes: anônimo recebe 401 na rota protegida; cliente A não consulta reserva de B; cliente não acessa KDS (403); criação pública com perfil admin resulta apenas em cliente ou é rejeitada; administrador executa operações autorizadas.

A02 — Cryptographic Failures (falhas criptográficas)

Risco no contexto do projeto. Senhas, identificadores de sessão e dados pessoais podem ser expostos quando transmitidos ou armazenados sem proteção apropriada.

Aplicação ao Sabor & Clic. Hoje a autenticação é demonstrativa e não há persistência real de senha. Na etapa final serão tratados nome, e-mail, telefone/endereço e credenciais.

Medidas previstas. Utilizar HTTPS na implantação; criar hash de senhas por mecanismo apropriado (Werkzeug generate_password_hash/check_password_hash ou bcrypt) com salt; armazenar SECRET_KEY em variável de ambiente; marcar cookies de autenticação como Secure, HttpOnly e SameSite; não registrar senhas e

evitar dados sensíveis em cookies ou URLs. O cookie padrão do Flask é assinado, não criptografado: guardar nele apenas dados mínimos.

Verificação e evidência. Inspeccionar a tabela de usuários sem encontrar senhas em texto puro; validar login e logout; checar flags dos cookies em ambiente HTTPS; confirmar ausência de segredos no Git e nos logs.

A03 — Injection (injeção)

Risco no contexto do projeto. Dados controlados pelo usuário podem alterar consultas SQL ou produzir execução indevida de conteúdo em HTML/JavaScript.

Aplicação ao Sabor & Clic. Campos de pesquisa, e-mail, observação do pedido e filtros de relatório serão entradas não confiáveis. O MySQL deverá receber DDL e consultas SQL construídos explicitamente pela equipe.

Medidas previstas. Usar prepared statements/queries parametrizadas em todos os valores; nunca concatenar entradas na SQL. Para nomes de colunas usados em ordenação, utilizar lista fechada de opções. No frontend, usar `textContent` para texto do usuário; no Jinja, manter `autoescape` e evitar marcar entradas como seguras. Aplicar CSP como defesa adicional, sem substituição da codificação da saída.

Verificação e evidência. Inserir entradas com aspas e símbolos especiais sem alterar a estrutura de consultas; tentar filtros malformados e observar erro seguro; testar uma observação com marcação HTML e confirmar sua exibição como texto, sem execução.

```
# Exemplo conceitual de valor parametrizado no MySQL:
cursor.execute(
    "SELECT id_reserva FROM reserva WHERE id_reserva = %s AND id_cliente = %s",
    (id_reserva, usuario_autenticado.id)
)
# No navegador: elemento.textContent = texto_fornecido;
```

A04 — Insecure Design (projeto inseguro)

Risco no contexto do projeto. Um sistema pode cumprir verificações de entrada e, ainda assim, permitir operações logicamente incompatíveis com as regras de negócio.

Aplicação ao Sabor & Clic. As regras do domínio incluem impedir reservas sobrepostas, vincular pedidos a sessões ativas, impedir estoques negativos e registrar preços históricos dos itens.

Medidas previstas. Definir invariantes de negócio; validar os estados de cada recurso no backend; utilizar transações ao criar pedidos e baixar estoque; impedir dupla confirmação; verificar concorrência na reserva e integridade dos vínculos; documentar fluxos indevidos (ex.: pedido para sessão encerrada).

Verificação e evidência. Testes: duas reservas para a mesma bancada/horário não coexistem; pedido rejeitado após encerrar sessão; baixa de estoque ocorre uma única vez; dados incompletos não deixam alterações parciais.

A05 — Security Misconfiguration (configuração insegura)

Risco no contexto do projeto. Modo debug, segredos fixos, permissões excessivas ou política permissiva de origem podem expor a aplicação.

Aplicação ao Sabor & Clic. O Flask é executado localmente na etapa demonstrativa; a implantação futura envolverá aplicação Web e MySQL.

Medidas previstas. Desligar debug em produção; usar SECRET_KEY forte obtida do ambiente; estabelecer configurações separadas de desenvolvimento e produção; conceder ao usuário de banco somente as permissões necessárias; limitar CORS às origens usadas; impor HTTPS; padronizar tratamento de erros e cabeçalhos de segurança; não publicar credenciais de exemplo como contas reais.

Verificação e evidência. Revisar configuração de implantação e cabeçalhos; confirmar ausência de debugger em produção e de senha/SECRET_KEY no repositório; verificar que origens não permitidas não têm acesso via navegador.

A06 — Vulnerable and Outdated Components (componentes vulneráveis e desatualizados)

Risco no contexto do projeto. Dependências com vulnerabilidades conhecidas podem introduzir risco mesmo quando o código da equipe está correto.

Aplicação ao Sabor & Clic. O backend utiliza Flask e sua árvore de dependências; o frontend pode empregar bibliotecas ou recursos externos de apresentação.

Medidas previstas. Fixar versões compatíveis no requirements.txt, revisar alertas de segurança e atualizar bibliotecas dentro de uma janela de manutenção; remover dependências não utilizadas; verificar a procedência de pacotes; manter o runtime Python e o MySQL suportados.

Verificação e evidência. Apresentar arquivo de dependências e resultado datado de auditoria (por exemplo, pip-audit); registrar correções efetuadas e executar os testes automatizados após atualizar.

A07 — Identification and Authentication Failures (falhas de identificação e autenticação)

Risco no contexto do projeto. Contas podem ser acessadas com credenciais inválidas, sessões podem permanecer válidas indevidamente e logins podem ser forçados por tentativas repetidas.

Aplicação ao Sabor & Clic. O login atual aceita qualquer senha para um e-mail demonstrativo e a role fica no localStorage; não existe sessão autenticada do servidor.

Medidas previstas. Comparar senha com hash; usar sessão autenticada com cookie protegido; regenerar ou renovar a sessão após autenticação; invalidar a sessão no logout; aplicar expiração por inatividade; limitar tentativas e registrar eventos relevantes; utilizar mensagens genéricas para login malsucedido. Nunca usar o papel armazenado no navegador como autorização.

Verificação e evidência. Testes: senha incorreta não entra; cookie ausente/expirado exige login; logout bloqueia reutilização da sessão; repetidas tentativas recebem limitação; mudanças de perfil só refletem após validação no servidor.

```
# Referência ilustrativa à API de hash do Werkzeug:  
from werkzeug.security import generate_password_hash, check_password_hash  
senha_hash = generate_password_hash(senha_recebida)  
autenticado = check_password_hash(senha_hash_salva, senha_digitada)
```

A08 — Software and Data Integrity Failures (falhas de integridade de software e dados)

Risco no contexto do projeto. Alterações não autorizadas nos artefatos do sistema ou em dados críticos comprometem resultados, mesmo sem roubo de credenciais.

Aplicação ao Sabor & Clic. Preços e totais enviados pelo navegador não serão confiáveis; o projeto terá scripts SQL, dependências, alterações de pedidos e estoque.

Medidas previstas. Recalcular preços e totais no backend a partir do cadastro e registrar preço histórico; controlar scripts SQL e alterações por Git/revisão; validar

dados de entrada antes de gravar; usar transações e restrições PK/FK/CHECK; proteger branches e revisar dependências e fluxos de implantação; evitar desserialização de dados não confiáveis com mecanismos inseguros.

Verificação e evidência. Testes: alterar preço no JSON não modifica o valor cobrado; inserções inválidas são revertidas; alterações nos scripts passam por revisão; backup/restauração preserva integridade referencial.

A09 — Security Logging and Monitoring Failures (falhas de registro e monitoramento)

Risco no contexto do projeto. Incidentes podem passar despercebidos quando falhas de login, negações de acesso e operações críticas não são registrados nem acompanhados.

Aplicação ao Sabor & Clic. Login, alterações administrativas, reservas, encerramento de sessão e atualização do preparo representam eventos de auditoria relevantes.

Medidas previstas. Produzir logs estruturados com data/hora, ID de correlação, identificador do usuário quando disponível, operação e resultado; não salvar senha, cookie completo, token ou dados pessoais desnecessários; centralizar logs em implantação e estabelecer alerta para falhas repetidas; controlar acesso e retenção de registros.

Verificação e evidência. Gerar casos de 401/403 e operações administrativas em teste; demonstrar eventos registrados sem segredos; documentar responsável pelo acompanhamento e tratamento dos alertas.

A10 — Server-Side Request Forgery (SSRF)

Risco no contexto do projeto. Uma funcionalidade que busca URLs arbitrárias pode ser induzida a acessar serviços internos ou endereços reservados.

Aplicação ao Sabor & Clic. As rotas atuais do Sabor & Clic não dependem de buscar URLs fornecidas pelo usuário. O risco passa a existir se forem introduzidas importação de imagens, webhooks, pré-visualização de links ou integrações externas.

Medidas previstas. Evitar requisições server-side para URLs arbitrárias. Se uma integração for necessária, usar lista de destinos aprovados, HTTPS, bloqueio de IPs internos e metadados, validação de DNS/redirecionamentos e limites de

tempo/tamanho de resposta. Nunca permitir que o cliente defina a URL de serviços internos.

Verificação e evidência. Caso seja adicionada funcionalidade de URL, testar rejeição de localhost e endereços privados, redirecionamentos proibidos e destinos fora da lista permitida. Se não houver uso de URL externa, registrar o controle como preventivo e condicional.

5 PROTEÇÕES COMPLEMENTARES E ORDEM DE IMPLEMENTAÇÃO

5.1 CSRF e XSS

Para operações que alteram estado e utilizam cookies de autenticação, será adotada proteção CSRF com token verificável no servidor (por exemplo, Flask-WTF/CSRFProtect), complementada por SameSite. O token será enviado em formulários ou no cabeçalho das chamadas JavaScript. Requisições GET não modificarão estado. Para XSS, textos fornecidos por usuários serão renderizados como texto ou escapados conforme o contexto de saída; não serão inseridos em innerHTML sem tratamento adequado.

5.2 Etapas e resultados esperados

Etapas	3º bimestre — documentação e protótipo	4º bimestre — implementação e evidências
Identidade RBAC	Perfis definidos; login explicitamente demonstrativo. Matriz por rota e CRUD; regras de propriedade.	Senhas com hash, sessão segura, logout e proteção contra força bruta. Decorators/middlewares e testes positivos/negativos por perfil.
Entradas Sessões/pedidos	Validações JS, regras e mensagens documentadas. Fluxo de negócio e invariantes registrados.	Validação de backend + SQL parametrizado + restrições no MySQL. Transações, concorrência, estados válidos e baixa de estoque.
Ambiente e auditoria	Plano OWASP e requisitos de configuração.	HTTPS, cookies seguros, configuração, logs e análise de dependências.

5.3 Casos mínimos de teste para apresentar

- Cliente A consulta `/api/reservas/<id>` de cliente B → negar, sem retornar dados de B.
- Cliente tenta enviar perfil admin no cadastro → negar ou forçar perfil cliente; registrar evento.
- Cozinheiro tenta alterar permissões de usuário → 403.

- Requisição sem login a /api/kds → 401; cozinheiro autenticado → acesso permitido.
- Pedido submetido para sessão encerrada ou para sessão alheia → rejeição sem persistência.
- Quantidade negativa, e-mail inválido e data/hora inconsistente → 400 com mensagem clara.
- Observação contendo marcação HTML → exibir texto, não executar script.
- Login com senha incorreta e várias tentativas → rejeição e limitação conforme política.
- Desligamento da sessão de login → cookie anterior não autoriza novas operações.
- Consulta SQL com caracteres inesperados → dados tratados como parâmetros; sem alteração da consulta.

6 LIMITAÇÕES CONHECIDAS DA VERSÃO ATUAL

Até esta entrega, o frontend usa localStorage para simular o perfil; o login da API não verifica senha; o cadastro aceita o perfil enviado pelo cliente; não há middleware de autorização, persistência SQL definitiva ou controles de CSRF/segurança de produção. Existem validações pontuais no JavaScript e regras de negócio no backend demonstrativo, mas essas verificações não equivalem à implementação integral de segurança. O objetivo do documento é explicitar as proteções futuras e os critérios pelos quais serão testadas.

7 NOTA DE VERSÃO DA OWASP

A versão OWASP Top 10:2025 é mais recente, mas a disciplina usa explicitamente os códigos de 2021. Em 2025, Injection passa a A05; Cryptographic Failures passa a A04; Security Misconfiguration passa a A02; Software Supply Chain Failures expande o tema de componentes; SSRF é incorporada ao grupo de controle de acesso; e surge uma categoria específica para tratamento incorreto de condições excepcionais. Para a avaliação, as dez seções acima mantêm a nomenclatura de 2021.

8 REFERÊNCIAS

Referências consultadas. Acesso em: 23 set. 2026.

OWASP. OWASP Top 10:2021. <https://top10.owasp.org/2021/>

OWASP. OWASP Top 10:2025. <https://top10.owasp.org/2025/pt-BR/>

OWASP. SQL Injection Prevention Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP. Session Management Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP. Cheat Sheet Series — Index Top 10.

<https://cheatsheetseries.owasp.org/IndexTopTen.html>

SABOR & CLIC. Repositório principal (main), documentação e código demonstrativo.

<https://github.com/FaellEmiliano/Sabor-Clic/tree/main>